

Symbiosis Institute of Technology, Pune

Faculty of Engineering

CSE- Academic Year 2025-26

Compiler Construction Lab Batch 2022-26

Lab Assignment No: - 10	
Name of Student	Pranav Suri
PRN No.	22070122152
Batch	22 -26
Class	B3
Academic Year & Semester	SEM VII
Date of Performance	
Title of Assignment	Intermediate Code Generation
Practice Questions	1. YACC program for IC generation for the expression involving parenthesis. 2. YACC program for IC generation for the expression involving programming constructs.
Source Code	1. YACC program for IC generation for the expression involving parenthesis. <u>LEX file:</u> <pre>%{ #include "y.tab.h" #include <stdlib.h> #include <string.h> }% digit [0-9]+ id [a-zA-Z_][a-zA-Z0-9_]* %% {digit}+ { yylval.ival = atoi(yytext); return NUM; } {id} { yylval.sval = strdup(yytext); return ID; } [\\t\\n]+ ; // ignore whitespace "=" { return '='; } "+" { return '+'; } "-" { return '-'; } "*" { return '*'; } "/" { return '/'; } ";" { return ';'; } "(" { return '('; } ")" { return ')'; }</pre>

```
. { return yytext[0]; }
%%
int yywrap() { return 1; }
```

YACC file:

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int temp_count = 0;
// Generate a new temporary variable
char* newTemp() {
    char* temp = (char*)malloc(5);
    sprintf(temp, "t%d", temp_count++);
    return temp;
}
// Generate intermediate code
void gen(char* op, char* arg1, char* arg2,
char* res) {
    printf("%s = %s %s %s\n", res, arg1, op,
arg2);
}
void yyerror(const char *s);
int yylex();
}%
%union {
    int ival;
    char* sval;
}
%token <ival> NUM
%token <sval> ID
%left '+' '-'
%left '*' '/'
%type <sval> expr term factor stmt
%%
prog: stmt_list ;
stmt_list:
    stmt_list stmt
    | stmt
    ;
stmt: ID '=' expr ';' {
    printf("%s = %s\n", $1, $3);
    free($1);
    free($3);
}
| expr ';' { free($1); }
;
```

```

expr:
    expr '+' term {
        char* t = newTemp();
        gen("+", $1, $3, t);
        free($1); free($3);
        $$ = t;
    }
    | expr '-' term {
        char* t = newTemp();
        gen("-", $1, $3, t);
        free($1); free($3);
        $$ = t;
    }
    | term { $$ = $1; }
    ;

term:
    term '*' factor {
        char* t = newTemp();
        gen("*", $1, $3, t);
        free($1); free($3);
        $$ = t;
    }
    | term '/' factor {
        char* t = newTemp();
        gen("/", $1, $3, t);
        free($1); free($3);
        $$ = t;
    }
    | factor { $$ = $1; }
    ;

factor:
    '(' expr ')' { $$ = $2; }
    | NUM {
        char* t = (char*)malloc(10);
        sprintf(t, "%d", $1); // $1 is ival
        $$ = t;
    }
    | ID { $$ = strdup($1); } // $1 is sval
    ;

%%

int main() {
    printf("Enter arithmetic expressions:\n");
    yyparse();
    return 0;
}

void yyerror(const char *s) {
    printf("Error: %s\n", s);
}

```

- }
2. YACC program for IC generation for the expression involving programming constructs.

LEX File:

```
%{
#include "y.tab.h"
#include <stdlib.h>
#include <string.h>
}%
digit [0-9]+
id [a-zA-Z_][a-zA-Z0-9_]*
%%
{digit}+ { yylval.ival = atoi(yytext);
return NUM; }
{id} { yylval.sval = strdup(yytext); return
ID; }
[\\t\\n ]+ ; // ignore whitespace
"=" { return '='; }
"+" { return '+'; }
"-" { return '-'; }
"*" { return '*'; }
"/" { return '/'; }
";" { return ';'; }
"(" { return '('; }
")" { return ')'; }
. { return yytext[0]; }
%%
int yywrap() { return 1; }
```

YACC File:

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int temp_count = 0;
// Create a new temporary variable
char* newTemp() {
    char* temp = (char*)malloc(5);
    sprintf(temp, "t%d", temp_count++);
    return temp;
}
// Generate intermediate code
void gen(char* op, char* arg1, char* arg2,
char* res) {
```

```

    printf("%s = %s %s %s\n", res, arg1, op,
arg2);
}
void yyerror(const char *s);
int yylex();
%}
%union {
    int ival;
    char* sval;
}
%token <ival> NUM
%token <sval> ID
%left '+' '-'
%left '*' '/'
%type <sval> expr term factor stmt
%%
prog: stmt_list ;
stmt_list:
    stmt_list stmt
    | stmt
    ;
stmt: ID '=' expr ';' {
    printf("%s = %s\n", $1, $3);
    free($1); free($3);
}
    | expr ';' { free($1); }
;
expr:
    expr '+' term {
        char* t = newTemp();
        gen("+", $1, $3, t);
        free($1); free($3);
        $$ = t;
    }
    | expr '-' term {
        char* t = newTemp();
        gen("-", $1, $3, t);
        free($1); free($3);
        $$ = t;
    }
    | term { $$ = $1; }
;
term:
    term '*' factor {
        char* t = newTemp();
        gen("*", $1, $3, t);
        free($1); free($3);
        $$ = t;
    }

```

```

    }
    | term '/' factor {
    char* t = newTemp();
    gen("/", $1, $3, t);
    free($1); free($3);
    $$ = t;
    }
    | factor { $$ = $1; }
    ;
factor:
    '(' expr ')' { $$ = $2; } // Handles
parentheses
    | NUM {
    char* t = (char*)malloc(10);
    sprintf(t, "%d", $1);
    $$ = t;
    }
    | ID { $$ = strdup($1); }
    ;
%%
int main() {
    printf("Enter arithmetic expressions:\n");
    yyparse();
    return 0;
}
void yyerror(const char *s) {
    printf("Error: %s\n", s);
}

```

Output Screenshot

```

~$ gcc lex.yy.c y.tab.c
~$ lex a10.l
~$ yacc a10.y -d
~$ gcc lex.yy.c y.tab.c
~$ ./a.out

```

```

Enter arithmetic expressions:
a = 5 + 2 * 3;
b = a - 4;
t0 = 2 * 3
t1 = 5 + t0
a = t1
t2 = a - 4
b = t2

```

```

~$ lex a10_2.l
~$ yacc a10_2.y
~$ yacc a10_2.y -d
~$ gcc lex.yy.c y.tab.c
~$ ./a.out

```

```

Enter arithmetic expressions:
x = (2 + 3) * (4 - 1);
t0 = 2 + 3
t1 = 4 - 1
t2 = t0 * t1
x = t2

```

Post lab questions

1. YACC program for IC generation for the expression involving programming constructs.

Lex File:

```

%{
#include "y.tab.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
}%
digit [0-9]+
id [a-zA-Z_][a-zA-Z0-9_]*
%%
{digit}+ { yylval.ival = atoi(yytext); return NUM; }
{id} { yylval.sval = strdup(yytext); return ID; }
"(" { return '('; }
")" { return ')'; }
"+" { return '+'; }
"-" { return '-'; }
"*" { return '*'; }
"/" { return '/'; }
"=" { return '='; }
[\t ]+ ; // ignore whitespace
[\n]+ { return '\n'; } // newline is treated as end of statement
. { return yytext[0]; }
%%
int yywrap() { return 1; }

```

```

~$ lex i.l
~$ yacc a.y -d
~$ gcc lex.yy.c y.tab.c
~$ ./a.out

```

	<pre>Enter program statements (press Enter after each statement, C trl+D to finish): a = 5 + 2 b = a * (3 + 4) c = b t0 = 5 + 2 a = t0 t1 = 3 + 4 t2 = a * t1 b = t2</pre>
Conclusion	Lab experiment was completed successfully.